

Feedback vertex sets through assignment-pair ordering

Reconstruction · awaiting expert review
22 September 2026

Abstract

We give deterministic polynomial-time construction and recovery maps from directed feedback vertex set to ordering simultaneous assignments with a threshold on conflicting assignment pairs. Vertex splitting represents each removable vertex by one dependency; private two-edge paths protect the source-arc relations. Every qualifying ordering recovers a source solution, even if it is nonoptimal or spends budget on path arcs. This is an executable reconstruction of established feedback-set ideas for an explicit pair-count objective, not a new reduction principle or a register-allocation result.

1. Context and result

Ordering simultaneous assignments can overwrite a value before a later assignment reads it. The fixed objective counts conflicting assignment pairs, rather than saved variables. A vertex-deletion source therefore requires more than one assignment per graph vertex: reversed dependencies must not substitute cheaply for deleted vertices.

Even et al. [1, §2.2, printed p. 6] reduce feedback vertex set to feedback edge set by splitting each vertex into incoming and outgoing parts. The connecting arc receives the vertex weight, and original arcs receive infinite weight. The backwards-edge ordering formulation is also discussed in [2, §§2–3.1]. We implement protected arcs with private paths and translate each graph arc into one assignment-pair conflict. The contribution is a self-contained realization and every-output decoder for this exact target. No priority claim is made for these ingredients or their composition.

Theorem 1. There are deterministic polynomial-time maps F and G such that, for every legal source instance x and every valid target output y ,

$$y \in S_B(F(x)) \implies G(x, y) \in S_A(x). \quad (1)$$

The output may be any qualifying ordering, not necessarily an optimal one, or the exact negative answer.

2. Problems and encodings

The source is a finite digraph $D = (V, A)$ with distinct explicitly listed integer labels, distinct arcs, and integer threshold k . Loops are allowed. A positive output is a set $S \subseteq V$ with $|S| \leq k$ such that $D - S$ is acyclic. If no such set exists, its only output is NO-SOLUTION. Numerical values have finite positional encodings and may be arbitrarily large or negative.

The target has assignments with distinct written variables, explicit read sets, and integer threshold K . An ordered pair contributes one conflict precisely when its earlier assignment writes a variable read by its later assignment. A positive output is a permutation of all assignments costing at most K ; otherwise the only output is NO-SOLUTION. Self-reads and reads of unwritten variables contribute no distinct pair. Empty structures are allowed. Both problems always have a valid output because infeasibility has an explicit negative answer.

3. Construction and recovery

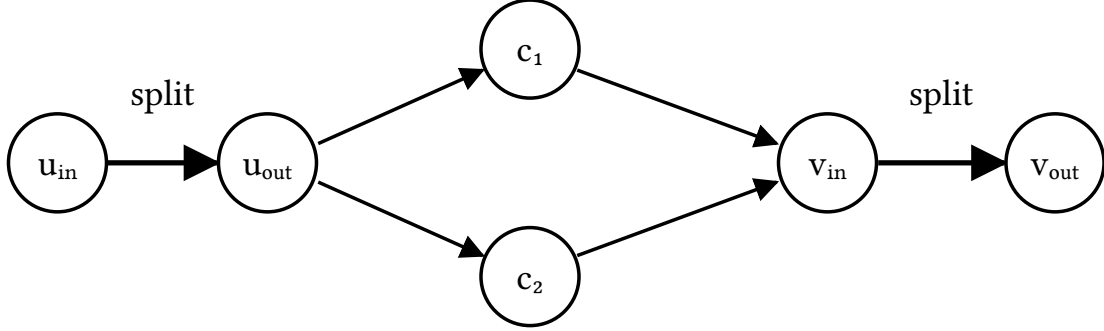
Write $n = |V|$ and $m = |A|$. For $k < 0$, emit an empty assignment list with $K = -1$. For $k \geq n$, emit an empty list with $K = 0$. These branches cover empty sources and avoid expansion proportional to a huge threshold. Otherwise set

$$0 \leq k < n, \quad b = k + 1 \leq n. \quad (2)$$

For each vertex v , create two vertices v_{in} and v_{out} with split arc $v_{\text{in}} \rightarrow v_{\text{out}}$. For each source arc $e = (u, v)$, create b private vertices and paths

$$u_{\text{out}} \rightarrow c_{e,j} \rightarrow v_{\text{in}} \quad (1 \leq j \leq b). \quad (3)$$

Call the resulting graph H . It is simple and loopless. A source loop creates a cycle through a split pair, not a self-read. Figure 1 gives the attachment for one source arc. Different gadgets share only their designated split vertices.



$k = 1$: two private paths protect the source arc $u \rightarrow v$.

Figure 1: A complete single-arc gadget for $u \rightarrow v$ at $k = 1$. Thick arcs are split arcs; two private middle vertices protect the endpoint relation. In a larger graph further paths attach at the split vertices and are omitted here. An arc $s \rightarrow t$ means that assignment s reads the variable written by t . Putting v_{in} before u_{out} incurs at least two conflicts.

Create one assignment per vertex s of H , writing a fresh variable w_s and reading exactly w_t for arcs $s \rightarrow t$. Set $K = k$. Number split pairs in source-list order, followed by private vertices in source-arc and copy order. This defines deterministic F with legal distinct writes and explicit read sets.

For a positive output in the expansion branch, G computes positions and returns

$$S = \{v \in V : v_{\text{out}} \text{ precedes } v_{\text{in}}\}. \quad (4)$$

In the branch $k \geq n$, it returns V . On NO-SOLUTION it returns NO-SOLUTION. The negative-threshold branch admits no positive target output. Neither map calls a solver.

4. Correctness

Lemma 2 (pair correspondence). Target cost equals the number of backwards arcs of H .

Proof. An arc $s \rightarrow t$ contributes exactly when t occurs earlier: its write of w_t then precedes a read by s . Conversely every conflict comes from such an arc. Distinct graph arcs give distinct assignment pairs. ■

Lemma 3 (protected endpoints). Every ordering of cost at most k puts u_{out} before v_{in} for every source arc $u \rightarrow v$.

Proof. Reversing these endpoints makes at least one arc backwards on each private two-edge path; otherwise the path itself forces the endpoints forward. These paths have disjoint arcs, so Lemma 2 gives at least $b = k + 1$ conflicts, exceeding the budget. This forces endpoint order only: individual path arcs may still be backwards. ■

Lemma 4 (every-order recovery). Equation 4 returns a feedback vertex set of size at most k from every qualifying ordering.

Proof. Each selected vertex contributes a distinct backwards split arc, so $|S|$ is at most the total cost. If a cycle survived in $D - S$, each of its incoming parts would precede its outgoing part. Lemma 3 puts that outgoing part before the incoming part of the next cycle vertex. Chaining the strict inequalities around the cycle is impossible, including for a one-vertex loop. The argument permits nonoptimal orders and extra backwards path arcs. ■

Lemma 5 (completeness). A source solution implies a qualifying target ordering.

Proof. Given a feedback vertex set S , remove only its split arcs from H . Any remaining cycle must alternate split arcs and two-edge source-arc paths: an incoming part can leave only on its split arc, and a private vertex has precisely its specified incoming and outgoing arcs. Its projection would be a source closed walk avoiding S , containing a cycle in $D - S$. Thus the remaining graph is acyclic. A topological ordering makes every retained arc forward. Only the at most $|S|$ removed split arcs can be backwards in the full graph, so Lemma 2 bounds its cost by k . ■

Proof of Theorem 1. Lemmas 4 and 5 prove every-positive-output recovery and equivalence of witness existence in the expansion branch. That equivalence also justifies negative-answer recovery. For $k < 0$, both instances are infeasible. For $k \geq n$, the empty target order recovers V , whose deletion leaves an empty acyclic graph. These branches include every empty-source case. The following bounds prove polynomiality. ■

5. Size and running time

The expansion branch has numbers of assignments and read incidences

$$N = 2n + bm, \quad M = n + 2bm. \quad (5)$$

Generated variable names use $O(\log(N + 2))$ bits, giving target length

$$O(1 + (N + M) \log(N + 2) + \log(k + 2)). \quad (6)$$

Since $b \leq n$, this is polynomial in the explicit input size. Construction uses $O(n + bm)$ elementary list and map operations with polynomial bit overhead for names, comparisons and label lookup. Even worst-case hash lookup remains polynomial. The constant branches compare the encoded threshold without expanding its value.

Recovery builds a position map and scans the split pairs, copying selected source labels. Its time and output length are polynomial in the combined source and target-output lengths. A canonical positive target witness has size $O(1 + N \log(N + 2))$. Integer conversion has polynomial bit cost, and the executable disables Python’s decimal digit ceiling. No fixed machine-integer bound is assumed.

6. Implications and limits

This reconstruction supplies a decoder for the fixed conflicting-pair objective, including loops and nonoptimal outputs. Historical priority of the private-path realization is unresolved. The expansion is $O(nm)$ and can be cubic on dense sources. The result asserts neither practical compiler performance, a new hardness classification, nor equivalence with minimizing saved variables.

References

- [1] G. Even, J. Naor, B. Schieber, and M. Sudan. *Approximating Minimum Feedback Sets and Multicuts in Directed Graphs*. Algorithmica 20(2), 151–174 (1998). Author manuscript, §2.2, printed p. 6.
- [2] A. Baharev, H. Schichl, and A. Neumaier. *An exact method for the minimum feedback arc set problem*. Author preprint, 12 December 2015. Primary text, §§1.1, 2, 3.1.

A. Verification and reproduction

The implementation and evidence are in `campaigns/feedback-vertex-set-parallel-assignments/work/`. After `uv sync --frozen` at the repository root, run the commands below from that work directory. The lockfile uses CPython 3.14.2 and z3-solver 5.1.0.0. The probe found `uv 0.12.7` and `Typst 0.15.1`. No Mathlib project or Lean proof is included.

```
uv run python algorithm.py < source.json
uv run python algorithm.py --extract < recovery.json
uv run python check.py --self-test
uv run python check.py --candidate algorithm.py
uv run python verify.py --candidate algorithm.py
cd paper
typst compile manuscript.typ manuscript.pdf
```

Forward input fields are vertices, arcs, and `k`. Recovery input fields are `source` and `target_solution`; a positive target output is `{"order": [...]}`, and a negative one is `{"status": "NO-SOLUTION"}`. Positive source outputs use `delete`. The exact domain is in `contract.md`; both modes are implemented in `algorithm.py`.

Prepare models source deletion and target pair counts independently in Z3. Its self-check covers ten source cases and 192 target thresholds, cross-checked with exhaustive orders, plus empty, huge-integer and malformed-output cases. The candidate run passed ten instances, nineteen recoveries and four negatives. Prepare commit `3dad512` precedes candidate implementation `fc8ebca`.

The separate verifier reads emitted assignments and solves them by subset dynamic programming, without importing the prepared checker. It obtains source ground truth by exhaustive deletion and transitive closure. Its finite domain includes all looped digraphs on at most two vertices at thresholds from minus one through the vertex count, all loopless three-vertex digraphs at thresholds zero and one, a three-cycle at threshold two, and huge-label boundaries. Results were 204 instances, 607 recoveries and 78 negative answers, including 116 nonoptimal feasible orders. It visited 1,091,028 subset states, with at most eighteen assignments per target. Finite tests do not replace the proof.

The registered fresh-context reviewer assessed revision `08314ee` and recommended advance as a reconstruction. Its additional checker exhausted 41,760 orders in three cases and validated all sixty feasible recoveries. Eight feasible orders spend budget on path arcs beyond split reversals. It also checked 6001-digit boundary integers. The report and checker are under the campaign's `reviews/001/` directory.

The records retain two process qualifications. An informal conceptual sketch preceded initialization and Prepare, although implementation followed the committed foundation. This does not erase that sequencing deviation. Prepare initially failed because Z3 could not infer the sort of an empty `Distinct` call. The exception was preserved, and the empty constraint was omitted without changing expected answers. No execution failure was treated as `NO-SOLUTION`. No solver or subprocess timeout is used.

Review used registered research-reviewer with fresh context and no model override. The resolved backend and per-agent usage were not exposed. Directory-only writes and no delegation were instruction restrictions, not an enforced sandbox; filesystem access was shared. Review is independent evidence, not formal certification or publication acceptance.